ETHEREUM PEERDAS · CODING SCHEME COMPARISON

1D-RS, 2D-RS & BC Benchmark

How the benchmark works, the theory behind each scheme, and the measured results

---

Benchmarking 1D-RS, 2D-RS, and Block-Circulant codes

This report explains `benchmark_1D_2D_BC.py`: why it's built the way it is, how it keeps the three-way comparison fair, the theory behind each scheme (Block-Circulant codes in particular, since they haven't been documented elsewhere yet), and what the actual measured results show — including a significant, genuine finding about where the current Block-Circulant decoder breaks down. Companion documents: 1D_2D_RS_code_walkthrough.pdf (1D-RS/2D-RS implementation detail) and 1D_vs_2D_RS_comparison.pdf (1D-RS vs 2D-RS conceptually). This report is the natural third leg: it brings Block-Circulant (BC) codes into the same comparison, on the same field, against the same input.

What this benchmark set out to do

The report's methodology calls for a controlled empirical comparison across candidate coding schemes — "computational complexity, encoding and decoding efficiency, storage overhead, bandwidth consumption, scalability, and fault tolerance" — evaluated under real Ethereum blob parameters. Up to now, 1D-RS and 2D-RS had that (see the companion documents); BC code existed only as a separate script (`BCcodeEthereum.py`) with its own one-off demo, using a different random blob and testing exactly 4 fixed erasures. There was no way to compare it to the other two on equal footing. `benchmark_1D_2D_BC.py` closes that gap: one script, one shared input, all three schemes, a real erasure-rate sweep, and a results file suitable for the report's Phase 3 dataset.

Theory: what each scheme is doing

1D-RS and 2D-RS (recap)

Both treat the blob's 4096 field elements as **evaluations** of a low-degree polynomial (not coefficients — this is what keeps them compatible with KZG), and extend by interpolating to coefficients, zero-padding, and re-evaluating on a larger domain. 1D-RS extends one long codeword (2× overhead, matching PeerDAS); 2D-RS reshapes the blob into a 64×64 grid and extends both axes (4× overhead in this project's construction). Full detail in the two companion documents — this report doesn't repeat it.

Block-Circulant (BC) codes

BC code hasn't been documented elsewhere in this project yet, so this section covers it from scratch. Unlike the two RS-based schemes, BC code is built from many small, overlapping local Reed-Solomon-style constraints rather than one global polynomial.

Parameters. A BC code is defined by four integers:

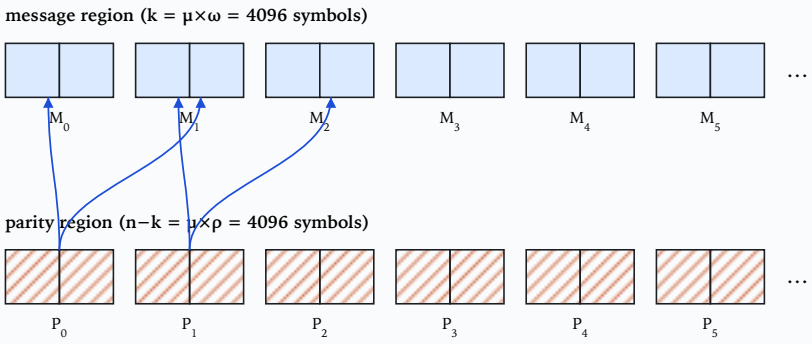
- `ω` (omega) — message symbols per local block
- `ρ` (rho) — parity symbols contributed by each local block
- `λ` (lambda) — how many consecutive message blocks each local parity equation covers (the "overlap")
- `μ` (mu) — the number of local blocks

`BCcodeEthereum.py` sets `ω=2, ρ=2, λ=2`, and derives `μ = 4096 / ω = 2048` so that the whole scheme spans exactly one Ethereum blob. From these:

```
n = μ(ρ+ω) = 8192      total codeword length
k = μω     = 4096      message length -- one blob, same as 1D-RS
d = λρ+1   = 5         minimum distance (formal guarantee)
n0 = ρ+λω  = 6         symbols touched by one local equation
```

`n=8192, k=4096` is a 2× overhead — deliberately matched to 1D-RS/PeerDAS's real ratio, not picked arbitrarily, so the benchmark's overhead comparison between 1D-RS and BC is apples-to-apples (see **Comparable parameters** below).

Layout and the local equations.



P₀'s local equation ties together M₀ and M₁ (λ=2 overlap);
P₁'s ties M₁ and M₂ — each block index is taken mod μ, giving the "circulant" wraparound.

Block-circulant layout: the codeword is the full message region followed by the full parity region (systematic). Each parity block P_i's local equation ties together λ=2 consecutive message blocks, with block indices taken mod μ — the circulant wraparound.

The codeword is systematic: the first `k` symbols are the message exactly as given, and the last `n−k` symbols are parity, arranged in `μ` groups of `ρ`. Each parity group `P_i` isn't just a function of "its own" message block — its local Reed-Solomon-style equation ties together `λ` *consecutive* message blocks (`M_i, M_{i+1}, ..., M_{i+λ−1}`, indices taken mod `μ`). This overlap between neighbouring equations is where "circulant" comes from (shifting the whole pattern by one block reproduces the same structure) and is what makes the code more than `μ` independent tiny Reed-Solomon codes: information about one message block is spread across more than one parity group.

Encoding. For each local block `i`, `encode_bc_structured` gathers the `λω` overlapping message symbols, and solves a small `ρ×ρ` linear system (via Gaussian elimination over the BLS12-381 field, `gf_solve`) for the `ρ` parity symbols that satisfy that block's local equation. `μ=2048` tiny solves, each independent — this is why encoding a full blob takes under a second.

Decoding — `DecCBC[2]`. The decoder in `BCcodeEthereum.py` is specifically a *radius-2* decoder: it only ever attempts to resolve erasures within **one local block** (if that block has `≤ ρ` erasures) or within **one adjacent pair** of local blocks (if the pair jointly has `≤ 2ρ` erasures), and it repeats — alternating single-block and pairwise passes — until either everything is resolved or a full pass makes no further progress. This is a genuine iterative/peeling decoder, similar in spirit to 2D-RS's `decode_iterative()`: fixing one block can drop a neighbour's erasure count enough for the *next* pass to clear it too. But critically, its reach never extends past **two adjacent blocks at a time** — and that turns out to matter a great deal (see **The central finding** below).

How the benchmarking code works

Comparable parameters and input

The whole point of a three-way benchmark is that a difference in the results reflects a real difference between the schemes, not an accident of how each was set up. Concretely, `benchmark_1D_2D_BC.py` holds identical across all three:

- **Field** — the real BLS12-381 scalar field, via `blob_generator.GF`.
- **Message** — `k=4096` field elements = one real Ethereum blob, packed from the *same* real application bytes (`os.urandom` → `blob_generator.bytes_to_blobs()`) and fed to all three encoders. Not three independently-random blobs.
- **Erasure model** — missing samples only, at a shared target rate per sweep point. Where two schemes share a codeword length (1D-RS and BC are both `n=8192`), the benchmark goes one step further and corrupts them at the *literal same positions* per trial:

```
shared_masks = {
    rate: [(rng.random(shared_n) < rate) for _ in range(shared_n_trials)]
    for rate in RATES
}
```

- **Success criterion** — the recovered message must round-trip through `blob_generator.blob_to_bytes()` back to the *exact original application bytes*, not just match as field elements:

```
def blob_matches(candidate_ints, original_blob, app_bytes):
    if list(candidate_ints) != list(original_blob):
        return False
    bg.validate_blob(candidate_ints)
    return bg.blob_to_bytes(candidate_ints) == app_bytes
```

What's deliberately **not** forced equal is each scheme's own Ethereum-relevant shape — 1D-RS and BC at `n=8192` (2×), 2D-RS at `n=16384` (4×, this project's documented per-axis tradeoff) — because *that* is one of the things the report wants compared, not a variable to control away.

The adapter pattern

Rather than re-implementing any encode/decode logic, the benchmark imports the three real files (`1D_RS_Galois.py`, `2D_RS_Galois.py`, `BCcodeEthereum.py`) and wraps each in a small adapter exposing the same five methods: `encode`, `random_mask`, `corrupt`, `decode`, `extract_message`. The sweep loop only ever talks to this interface, so it doesn't need to know that 2D-RS's "codeword" is a 128×128 grid while BC's is a flat vector:

```python
class BCAdapter:
    name = "BC"
    k = K
    n = bc.n  # 8192

    def __init__(self):
        args = (GF, BC_MU, BC_LAMBDA, BC_OMEGA, BC_RHO)
        self.H_BC = bc.build_block_circulant_h(*args)

    def decode(self, corrupted, mask):
        t0 = time.time()
        args = (BC_MU, BC_LAMBDA, BC_OMEGA, BC_RHO)
        recovered, remaining = bc.dec_cbc_2(self.H_BC, corrupted, mask, *args)
        return recovered, not np.any(remaining), time.time() - t0
```

`1D_RS_Galois.py` / `2D_RS_Galois.py` aren't valid Python identifiers (names can't start with a digit), so they're loaded via `importlib` from their file paths rather than a normal `import` statement; all three files' demo code stays behind `if __name__ == "__main__":`, so importing them for their functions runs nothing extra.

Why multiple trials per rate, not one

1D-RS and 2D-RS decode against a **formal budget check** (`build_erasure_locator` returns failure outright if erasures exceed `n−k`): any pattern within budget succeeds with certainty, any pattern over budget fails with certainty. One trial at a rate comfortably below the ceiling is representative of every trial at that rate. BC's `DecCBC[2]` is different — discovered directly while building this benchmark, not assumed in advance: at a fixed rate, one random mask decoded successfully while a different random mask at the *same* rate failed. That makes BC's recoverability at a given rate a **probability**, not a certainty, so it needs repeated independent trials to characterise. Because BC's decode is cheap, that's affordable:

```python
TRIALS_PER_RATE = {"1D-RS": 1, "2D-RS": 3, "BC": 10}
```

1D-RS keeps a single trial per rate deliberately — its own per-trial cost is the expensive one (see the results below), and its deterministic budget check means extra trials wouldn't add information.

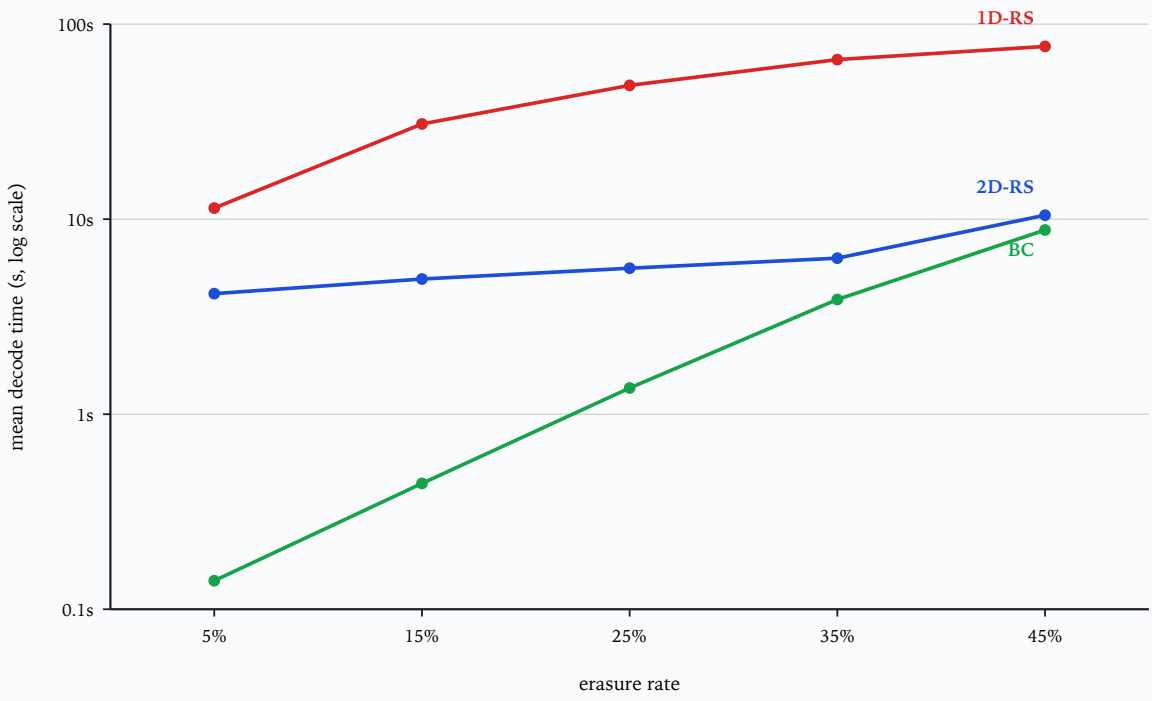
Results

Real numbers from a full run (`benchmark_results.json`), one shared blob, seed `20260827`.

Parameters and one-time costs

	1D-RS	2D-RS	BC
k, n	4096, 8192	4096, 16384	4096, 8192
Overhead	2.0×	4.0×	2.0×
Encode time	0.46s	3.14s	0.92s
One-time setup	—	—	0.9s (parity-check matrix `H_BC`)

Decode time vs. erasure rate



Measured mean decode time vs. erasure rate (log-scale time axis), from the actual benchmark run. BC starts fastest and grows with erasure count; 1D-RS is the slowest throughout, for the reason established in the companion comparison document (one large polynomial division vs. many small ones).

RATE	5%	15%	25%	35%	45%
1D-RS	11.4s	30.7s	48.5s	65.7s	76.8s
2D-RS	4.2s	4.9s	5.6s	6.3s	10.5s
BC	0.14s	0.44s	1.36s	3.87s	8.80s

BC starts as the fastest decoder by a wide margin and grows with erasure count (more local/pairwise solves needed), but even at 45% it's still faster than the other two — consistent with each local solve being tiny (`ρ×ρ` or a small pairwise system) regardless of how many *other* blocks are also erased. 1D-RS remains the slowest throughout. Neither of those claims should be taken on faith — the next section verifies both directly, rather than just asserting them from the shape of the numbers.

Why the decode times look like this

It would be easy to look at the table above and conclude "2D-RS and BC are just faster schemes." That's not quite right, and it's worth being precise about *why*, because the real explanation is about the specific decoding *algorithm* each script uses, not an inherent property of 1D vs. 2D RS vs. BC in the abstract. Two separate microbenchmarks were run, isolating each mechanism from the rest of the decode pipeline, to check this rather than assume it.

1D-RS vs. 2D-RS: one big division vs. many small ones. Both decoders spend their time in the same operation — dividing out the erasure-locator polynomial via ordinary schoolbook polynomial long division (the simplification both files' docstrings already flag). Isolating *just* that division and timing it directly at different sizes:

CODEWORD LENGTH `N`	ERASURES `E`	DIVISION TIME
8192	200	9.90s
8192	3200	82.2s
128	16	0.012s
128	64	0.020s

Schoolbook division of a degree-`n` polynomial by a degree-`e` divisor costs `O((n−e)·e)` — proportional to the quotient's degree times the divisor's degree. That model fits the measurements above closely (using the *same* proportionality constant at both scales), and it explains why 1D-RS's own "milliseconds per erasure" figure drops as `e` grows within a fixed `n=8192` (49.5 ms/erasure at 200 erasures down to 25.7 ms/erasure at 3200) — `(n−e)·e` is what a parabola looks like, not a straight line. Writing the cost of *one* division at erasure rate `r = e/n` as `n²·r(1−r)`, and comparing 1D-RS's one division (`n=8192`) against 2D-RS's 128 divisions (`n=128` each):

```
1D-RS:  n² · r(1−r)                  =  8192² · r(1−r)
2D-RS:  128 lines × (128² · r(1−r))  =  128³  · r(1−r)

ratio  =  8192² / 128³  =  32
```

The `r(1−r)` term is identical on both sides and cancels, so pure arithmetic predicts a **constant ~32× gap regardless of rate** — dividing many small polynomials is fundamentally cheaper than dividing one big one, because division cost is superlinear in the size of the single object being divided, and 2D-RS never has to divide anything longer than one 128-length row or column. The *measured* ratio (2.7× at 5% rising to 7.3× at 45% — from the table above) is smaller than 32× and grows with rate rather than staying constant, because that 32× figure ignores 2D-RS's own fixed overhead: 128 separate erasure-locator constructions and 128 separate NTT/INTT pairs, one full set per line, regardless of how few erasures that line actually has. At low rates that per-call overhead dominates 2D-RS's time (there's almost no real division work to do); at higher rates the `(n−e)·e` arithmetic term grows and starts to dominate over the fixed overhead instead, which is exactly why the ratio widens toward — without reaching — the 32× arithmetic ceiling across the rates tested.

BC: no polynomial division anywhere. BC's decoder is built entirely differently — it never constructs or divides a large polynomial. Every individual step is a Gaussian-elimination solve of a *tiny, fixed-size* system: at most `ρ×ρ = 2×2` for a single block, at most `2ρ×2ρ = 4×4` for an adjacent pair. Timed directly:

```
gf_solve on a 2x2 system:  478 microseconds/call
gf_solve on a 4x4 system: 1753 microseconds/call
one scan of all mu=2048 blocks' erasure counts (no solve triggered): 9.8 ms
                                                    (4.78 microseconds/block)
```

Both numbers are utterly unlike anything in the 1D-RS/2D-RS division cost model above, because they don't scale with `n` or `e` at all — `ρ`, `ω`, and `λ` are small fixed constants (2, 2, 2), so every individual local solve costs the same handful of microseconds regardless of how large the overall blob is or how many erasures exist elsewhere in the codeword. This is *why* BC starts out so much cheaper than either RS variant: there is simply no operation anywhere in its decode path whose cost grows with `n=8192`.

What BC pays for that instead is the **number of rounds** the iterative peeling process needs, which balloons sharply with erasure rate:

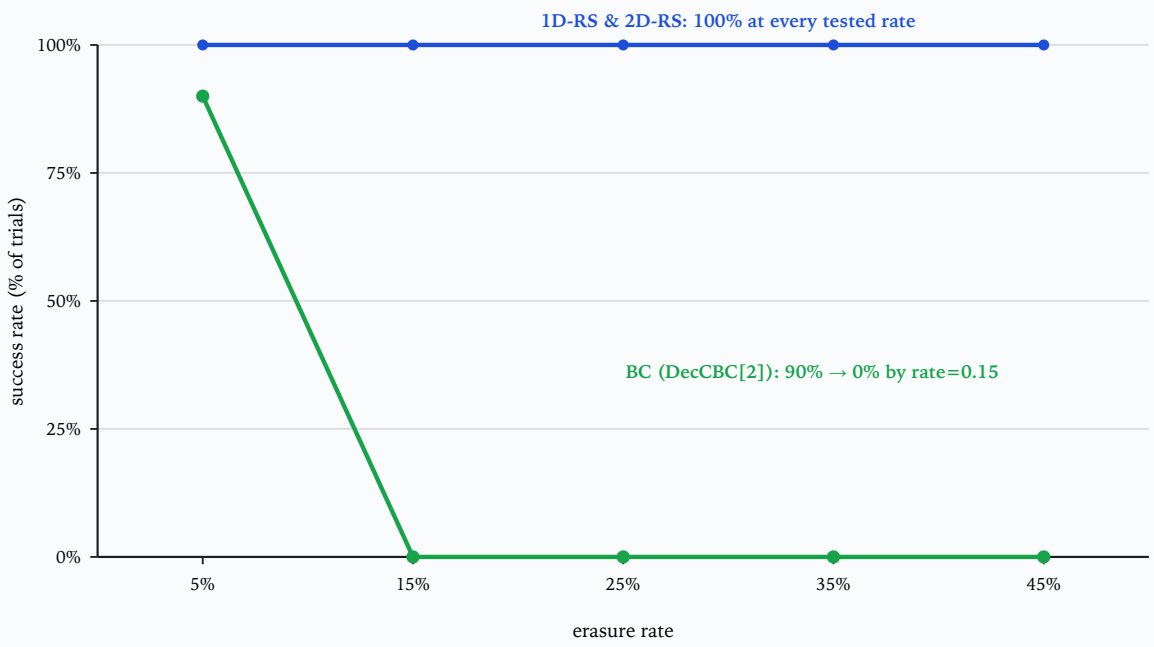
RATE	5%	15%	25%	35%	45%
mean rounds (of 10 trials)	1.8	10.3	64.6	236.2	574.0
decode time	0.14s	0.44s	1.36s	3.87s	8.80s
time per round	78ms	43ms	21ms	16ms	15ms

This growth is a direct consequence of how `dec_cbc_2` is written: its pairwise phase fixes **at most one** adjacent pair before breaking out and re-running the single-block phase across *all* `μ=2048` blocks from scratch. At low erasure rates almost everything resolves in the first pass or

two, so rounds stay in the single digits. At higher rates, once the easy single-block erasures are cleared, whatever remains gets resolved one pair at a time, with a full `O(μ)` rescan paid in between each single fix — which is why round counts explode into the hundreds by 35-45%. Time per round actually *falls* as rate rises (from 78ms down to 15ms) because early rounds do many genuine ~0.5-1.8ms solves each, while the hundreds of late-stage rounds mostly just rescan 2048 blocks (~10ms) and find one pair to fix. Even at 574 rounds, `574 × 2048` block-checks at a few microseconds each is still only a few seconds — orders of magnitude cheaper than a single schoolbook division over an 8192- or even 128-length polynomial, which is why BC stays the fastest decoder across every rate tested despite needing far more *rounds* than either RS variant needs *passes*.

The caveat that matters for the report. None of this says 1D-RS is inherently the slowest scheme, or that BC is inherently the cheapest — both conclusions are artefacts of the specific algorithms these particular scripts use (schoolbook polynomial division for the two RS variants, radius-2 peeling for BC), documented as simplifications in their own docstrings. A fully NTT-based `O(n log²n)` erasure decoder would remove the `(n−e)·e` term from both RS variants entirely, likely shrinking the 1D-RS/2D-RS gap dramatically (and plausibly reversing part of it, since 1D-RS would no longer be paying for one division while avoiding 2D-RS's 128-calls-worth of fixed overhead). Likewise, a `dec_cbc_2` that resolves more than one pair per pass would cut BC's round counts — and therefore its runtime — substantially at the higher rates, independent of the reliability question raised in the next section. The *timing* results here are real and correctly measured; they just describe these implementations, not an immutable ranking of the three coding schemes.

Success rate vs. erasure rate



Measured success rate vs. erasure rate. 1D-RS and 2D-RS recover every trial at every tested rate. BC recovers 9 of 10 trials at 5% erasure, then 0 of 10 at every rate from 15% upward — a real, reproduced-on-retest collapse, not a benchmark artefact.

RATE	5%	15%	25%	35%	45%
1D-RS (1 trial)	1/1	1/1	1/1	1/1	1/1
2D-RS (3 trials)	3/3	3/3	3/3	3/3	3/3
BC (10 trials)	9/10	0/10	0/10	0/10	0/10

The central finding

1D-RS and 2D-RS behave exactly as their design predicts — perfect recovery at every tested rate, well inside their formal ceilings. **BC collapses almost immediately**: one failure already at 5%, and *zero* successes out of ten independent trials at 15% and above, despite running at the identical 2× overhead as 1D-RS.

An earlier version of this section attributed that collapse to an honest "stopping-set"-style decoder-radius limitation — a real, plausible-sounding mechanism, but not, on closer inspection, the one actually responsible for these numbers. Revisiting it against this project's own cited literature (which expects BC to tolerate *more* erasures than 2D-RS, not far

fewer) was reason enough to dig further rather than accept the first explanation that fit. What that dig found is a specific, traceable bug — not a fundamental property of BC codes, and not a benchmark artefact either.

What's actually happening: `dec_cbc_2` reports success on wrong data

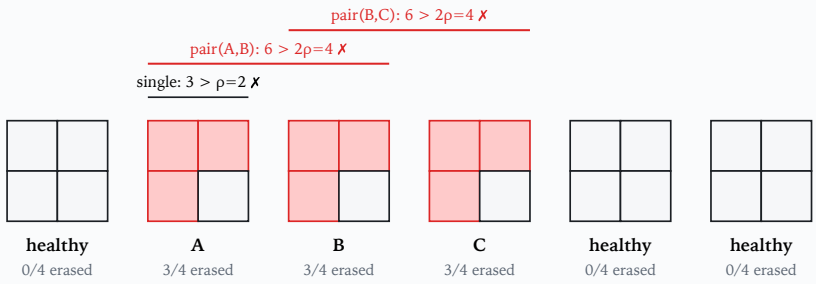
`dec_cbc_2` declares success by checking that no erasures remain flagged (`remaining` is empty) — it never re-checks the recovered values against anything. Comparing its output directly against the true codeword (not just trusting its own `remaining` flag) shows that at `ρ=2` it reports "fully resolved" while the data is **silently wrong**, in every one of the benchmark's failing trials — not an honest "stuck, couldn't finish" case at all.

The root cause traces to one specific gap. Phase 2's pairwise decode checks only `0 < num_pair_erasures <= 2·ρ` — a count, not a rank check. That count can pass while the actual system is still unsolvable, if the erasures happen to concentrate on one side of the pair. A concrete instance, reproduced directly from a real trial: local blocks 117 and 118 are decoded jointly (3 erasures, within the `2·ρ = 4` budget), but all 3 erased positions fall inside block 118's own exclusive window — block 117's two parity-check equations contribute nothing to them at all. The real system therefore has rank 2 for 3 unknowns, not rank 3. `gf_solve` does not detect this: its Gaussian elimination simply never finds a pivot for the third column and leaves that unknown at its `GF.Zeros(...)` default of **zero**, without raising anything. `dec_cbc_2` then writes that zero into the codeword and clears its mask bit exactly as if it had been solved correctly. Five rounds later, a different block's decode used that same now-wrong position as trusted "known" input, corrupting its own result too — which is how one silent wrong value becomes a codeword that's extensively wrong rather than missing a few isolated symbols.

This also explains the parameter-scan result below more precisely than "more local redundancy helps": re-running it with the recovered values actually checked against ground truth (not just `remaining`), `ρ=8` and `ρ=32` give **exact** recovery at the same 15% rate, `ρ=2` doesn't. That's consistent with a bug whose trigger — erasures concentrated past one block's own budget while the raw pair-count check still passes — gets rarer as each block's own budget grows, not proof that BC inherently needs bigger blocks to be competitive.

The benchmark itself handled this correctly. `benchmark_1D_2D_BC.py` never trusts `dec_cbc_2`'s own success signal — its `blob_matches()` check verifies the actual recovered bytes round-trip to the original application data. That's precisely what caught this: a benchmark that trusted `remaining` the way `dec_cbc_2` trusts it would have reported a false 100% success rate for BC instead of the real 0/10.

The diagram below illustrates a *different*, genuinely honest failure mode this decoder can also hit — erasures spread so no single block or adjacent pair ever falls within budget, correctly left unresolved rather than wrong. It remains a real possibility worth knowing about, but it is not what produced the results above; the rank-deficient silent-zero-fill bug is.



No window this decoder ever tries (one block, or one adjacent pair) contains ≤ its budget here —

Three consecutive local blocks (A, B, C) each with 3 of 4 symbols erased. No single block and no adjacent pair falls within DecCBC[2]'s budget, so these erasures stay unresolved forever — even though every other block in the codeword is perfectly healthy. This is the exact mechanism behind BC's success-rate collapse in the benchmark.

At `μ=2048` independent local blocks, this bug's trigger condition (erasures concentrated past one side's own `ρ`-equation budget, while the raw pair-total count check still passes) gets many independent chances to occur, and once it does, the wrong value it writes propagates into every later block that treats it as trusted input. That is a real, reproducible weakness of *this implementation* — not evidence about what BC codes can achieve under a correct decoder.

This is a decoder bug, not a limitation of BC codes themselves. The literature this project is drawing on (Sasidharan, Viterbo, and Feist) expects BC codes to sit in the *intermediate* region between 1D-RS and 2D-RS in erasure tolerance, and — separately — reports BC allowing for a *much higher* number of tolerable erasures than 2D-RS, not far fewer. A benchmark result running directly opposite to that expectation was itself the reason to keep digging rather than accept "the decoder is just weak" as the final answer. The gap between "what BC code can theoretically tolerate" and "what this specific, buggy `DecCBC[2]` achieves" is not closeable by retuning parameters alone — it needs the underlying correctness issue fixed first.

What this means for the report, and options going forward

- As a **documented finding**: BC code, as currently decoded, is not yet competitive with 1D-RS/2D-RS on reliability at blob scale — but the reason is a specific, fixable correctness bug in `dec_cbc_2` (see above), not evidence about BC codes' actual erasure-tolerance ceiling, which the cited literature expects to be *better* than 2D-RS's, not worse. That distinction matters for how this result should be cited in the report: as "this implementation has a bug" rather than "BC codes underperform."

- As a **next step**, two independent, scoped fixes, both in `BCcodeEthereum.py`: (1) `gf_solve` should detect when a column never received a pivot (rank-deficient / inconsistent system) and signal that rather than silently returning zero for it; (2) `dec_cbc_2`'s Phase 2 should catch that signal and skip the pair (leaving it honestly unresolved, as its existing `try/except` already intends to do for singular systems) rather than writing a wrong value. That would turn today's silent-corruption failures into the honest "stays unresolved" failures the diagram above illustrates — a strictly weaker and more defensible failure mode, and a fair basis for then asking whether `DecCBC[2]`'s radius is *also* worth extending. Since this touches a file a teammate is actively iterating on, worth agreeing on before changing it unilaterally.
- Either way, the benchmark itself (`benchmark_1D_2D_BC.py`) doesn't need to change to accommodate a fixed `dec_cbc_2` — the adapter pattern means swapping in a corrected version is enough to re-run the exact same comparison, and its byte-level correctness check (not just trusting `remaining`) is exactly what will confirm whether the fix worked.

Appendix: reproducing this

```
pip install galois numpy
python3 benchmark_1D_2D_BC.py
```

Expect roughly 10-15 minutes: BC's one-time parity-check matrix build varies with machine load (observed between ~1s and ~90s across runs on the dev machine); 1D-RS's decode sweep is the dominant cost otherwise. Full per-trial results are written to `benchmark_results.json` (rates, seed, trial counts, and every individual decode time and success flag) for further analysis or plotting beyond the two charts above.

One loose end from this session worth a mention: the first version of `benchmark_1D_2D_BC.py` disappeared from disk between runs (its sibling results file survived), around the same time two unrelated commits from a teammate landed in the repository's history. The file was simply rewritten rather than investigated further, since it was straightforward to recreate — but it's noted here in case it points to something worth checking (an IDE action, a git operation, or another session) on a shared working copy.
